A

MINI PROJECT REPORT

ON

WEB – BASED SMART ATTENDANCE CALCULATOR

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

K. AANUJ REDDY (22R91A05C7**)**

Under the Guidance

Of

P. RATNA TEJASWI

(Asst.Professor)

Department of CSE



Department of Computer Science and Engineering

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(An Autonomous Institution)

Medbowli, Meerpet, Saroornagar, Hyderabad – 500097

(Affiliated to JNTUH, Approved by AICTE, Accredited by NBA & NAAC) (2022-2026)

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(Sponsored by TKR Educational Society)
Approved by AICTE, Affiliated by JNTUH, Accredited by NBA & NAAC)
Medbowli, Meerpet, Saroornagar, Hyderabad – 500 097.
Phone: 040-24092838 Fax: +91-040-24092555
E-mail: tkrec@rediffmail.com Website: www.tkrec.ac.in

Department of Computer Science & Engineering

College code: R9

CERTIFICATE

This is to certify that the Mini Project report on ("**Web-Based Smart Attendance Calculator**") is a bonafide work carried out by **K. AANUJ REDDY (22R91A05C7)** in partial fulfillment for the requirement of the award of B.Tech degree in Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Hyderabad, affiliated to Jawaharlal Nehru Technological University, Hyderabad under my guidance and supervision.

The result of investigation enclosed in this report have been verified and found satisfactory. The results embodied in the project work have not been submitted to any other University for the award of any degree.

INTERNAL GUIDE

P. Ratna Tejaswi

(Asst.Professor)

……..

HEAD OF DEPARTMENT

Dr.CH.V.Phani Krishna

Professor

……..

EXTERNAL EXAMINER

……..……

PRINCIPAL

Dr. K. Venkata Murali Mohan

Professor

……

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(Sponsored by TKR Educational Society)
Approved by AICTE, Affiliated by JNTUH, Accredited by NBA & NAAC)
Medbowli, Meerpet, Saroornagar, Hyderabad – 500 097.
Phone: 040-24092838 Fax: +91-040-24092555
E-mail: tkrec@rediffmail.com Website: www.tkrec.ac.in

Department of Computer Science & Engineering

College code: R9

DECLARATION

I hereby declare that the Mini Project report entitled **"Web-Based Smart Attendance Calculator"** is done under the guidance of **Mrs. P. RATNA TEJASWI**. Assistant professor**,** Department of Computer Science and Engineering, Teegala Krishna Reddy Engineering College, is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from **Jawaharlal Nehru Technological University**, Hyderabad.

This is a record of bonafide work carried out by me in **Teegala Krishna Reddy Engineering College** and the results embodied in this project have not been reproduced or copied from any source.

Submitted by

K. AANUJ REDDY (22R91A05C7)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragementand guidance have crowned our efforts with success.

I extend my deep sense of gratitude to **Dr.K.Venkata Murali Mohan**, **Principal** Teegala Krishna Reddy Engineering College, Meerpet, for permitting me to undertake this project.

I am indebted to **Dr.CH.V.Phani Krishna, Professor & Head of the Department**, Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Meerpet forhis support and guidance throughout our project.

I am indebted to our guide **P. SWETHA**, Assistant professor Computer Science Engineering, Teegala Krishna Reddy Engineering College, Meerpet for his/her support and guidance throughout our project.

Finally, I express thanks to one and all that have helped me in successfully completing this Mini project. Further I would like to thank my family and friends for their moral support and encouragement.

By

K. AANUJ REDDY (22R91A05C7)

ABSTRACT

Attendance management is a critical aspect of academic success, yet students often struggle to keep track of their attendance progress and maintain the required percentage. This project presents a Web-based Smart Attendance Calculator designed to help students monitor and plan their attendance effectively. The tool allows users to input their total classes, attended classes, and target attendance percentage. Additionally, students can specify the number of working days left until their exams to receive actionable insights on how many classes they need to attend or can afford to miss to achieve their desired attendance. It features an intuitive interface built using HTML, CSS, and JavaScript, ensuring accessibility across devices. This project aims to empower students to make informed decisions about their attendance, minimize the risk of falling below the required threshold, and ultimately contribute to academic accountability and success.

The Smart Attendance Calculator allows users to input essential data, including the total number of classes conducted, the number of classes attended, their target attendance percentage, and the number of working days left before exams. Based on these inputs, the tool calculates whether students need to attend all remaining classes or if they can afford to miss a few while still maintaining the desired attendance level. Developed using HTML, CSS, and JavaScript, the calculator features an intuitive and responsive interface that works seamlessly across various devices. This project not only simplifies attendance tracking but also promotes student responsibility and academic accountability, enabling learners to stay on top of their goals and avoid unnecessary academic risks.

LIST OF FIGURES

F.NO	NAME	PAGE NO
4.1	SYSTEM ARCHITECTURE	7
4.2.1	CLASS DIAGRAM	9
4.2.2	USE CASE DIAGRAM	10
4.2.3	ACTIVITY DIAGRAM	11
4.2.4	SEQUENCE DIAGRAM	12
7.1	LOGIN PAGE	29
7.2	ROLE SELECTION	29
7.3	ADMIN DASHBOARD	30
7.4	STUDENT ATTENDANCE CALCULATOR	31
7.5	ATTENDANCE CALCULATOR	32
7.6	FINAL OUT-COUNT	33

CONTENTS

S.no	TITLE	PAGE NO
1	**Introduction**	**1**
2	**Literature Survey**	**2**
3	**System Analysis**	**4**
	1. Existing System	4
	2. Proposed System	4
	3. System Requirements	5
	1. Software Requirements	5
	2. Hardware Requirements	5
4	**System Design**	**6**
	1. System Architecture	7
	2. Uml Diagrams	7
	1. Class Diagram	9
	2. Usecase Diagram	10
	3. Activity Diagram	11
	4. Sequence Diagram	12
5	**System Implementation**	**13**
	1. Node.js and express.js	13
	2. SQLite Database	14
	3. HTML, CSS, js(frontend)	14
	4. Visual Studio Code	15
	5. Windows	15
	6. Modules	16
6	**Sample Code**	17
	1. login.html	17
	2. student.html	19
	3. admin.html	21
	4. dashboard.css	24
	5. server.js	25
	6. database.js	27
7	**Output Screens**	**29**

1. Login page 29
2. Role selection 29
3. Admin dashboard 30
4. Student attendance calculator 31
5. Attendance calculation 32
6. Final out-count 33

8 Conclusion 34

9 Reference 35

1.INTRODUCTION

Attendance tracking is a crucial aspect of educational institutions. Traditionally, attendance is recorded manually, which is time-consuming and prone to errors. The Attendance Calculator automates this process, providing students with real-time attendance insights. This project will assist both students and faculty in maintaining accurate attendance records and ensuring compliance with minimum attendance requirements. To achieve this discipline, it is essential to monitor students and keep a track of their devotion to their work. Web based Attendance Management is one such method that allows the admin to keep an eye on the presence and absence contribution of students.

One of the standout features of a smart attendance calculator is its ability to support goal-oriented planning. Students can set a target attendance percentage—such as 75% or 85%—and the system can suggest how many more classes they need to attend or how many they can afford to miss without falling below the threshold. This empowers students to manage their schedules responsibly and avoid potential academic consequences due to low attendance.

Because it is web-based, the calculator can be accessed from any device with an internet connection, including smartphones, tablets, and laptops. This ensures flexibility and convenience for users on the go. The user interface is typically designed to be interactive and intuitive, allowing users to simulate different attendance scenarios with ease. For example, they can explore the impact of missing future classes or attending all remaining sessions on their final attendance percentage.

From a technical perspective, such calculators are often built using modern web technologies. The frontend may use HTML, CSS, and JavaScript frameworks like React to ensure a responsive and engaging user experience. Overall, a web-based smart attendance calculator is a practical and intelligent solution that benefits both students and educators.

2.LITERATURE SURVEY

Attendance management has long been a critical component of educational institutions, serving both as a measure of student engagement and a factor in academic eligibility. Traditionally, attendance has been tracked using manual systems such as paper registers, roll calls, or spreadsheet-based tracking methods. While simple to implement, these systems are often riddled with inefficiencies, including the potential for human error, data loss, manipulation, and the time-consuming nature of manually entering or updating records. As institutions grow in size and scale, the limitations of manual methods become increasingly evident, prompting a shift toward more automated and digital solutions.

Over the past decade, various automated attendance systems have been proposed and implemented to overcome these challenges. Biometric-based systems, such as fingerprint scanners and iris recognition tools, have gained popularity in large-scale deployments. These systems reduce the possibility of proxy attendance and offer a more secure method of verification. RFID (Radio Frequency Identification)-based systems are also widely used, where students swipe ID cards to register attendance. However, both these technologies require dedicated hardware, regular maintenance, and can be expensive to deploy—making them less viable for smaller institutions or budget-conscious environments.

On the other end of the spectrum are basic web-based attendance tools or Excel-based systems. These offer some level of digitalization but still rely heavily on manual data entry and lack interactivity, real-time feedback, and student-side transparency. While they reduce paper usage and centralize data to an extent, they do not adequately support scalability, automation, or analytics. Moreover, most of these systems are admin-centric, offering no real-time visibility to students regarding their attendance status, nor any predictive insights regarding their attendance goals or eligibility.

Given these limitations, the need for a flexible, role-based, web-based attendance management system becomes evident. The proposed project

addresses this need by implementing a lightweight, yet fully functional attendance tracking application. The system features distinct dashboards for administrators and students. Admins can mark attendance, view the entire student list, and update records, while students can log in to view their personal attendance details. Additionally, the system introduces an innovative component: an attendance calculator that allows students to predict how many classes they can afford to miss or how many more they need to attend to meet a desired attendance threshold. This predictive analytics feature, commonly absent in most traditional systems, empowers students with better academic planning tools.

The system is built using modern web technologies including HTML, CSS, and JavaScript for the frontend, Node.js with Express for the backend server, and SQLite for lightweight, persistent database storage. The choice of SQLite ensures easy deployment without the need for complex server configurations, while still offering full SQL capabilities. The database structure is designed to store essential information such as student IDs, names, number of classes attended, and total classes conducted. The backend is structured to follow a three-tier architecture, separating presentation, logic, and data storage layers, improving modularity and maintainability.

Literature on educational technologies and digital transformation in classrooms also supports the effectiveness of student-facing dashboards. Research has shown that when students are provided with regular updates about their performance metrics, such as attendance, engagement improves significantly. Moreover, systems that provide predictive tools (like calculators and alerts) help students make informed academic decisions, which can positively influence retention and performance.

In summary, this literature survey highlights the need for a middle-ground solution: one that lies between outdated manual tracking systems and high-end biometric hardware solutions.

3.SYSTEM ANALYSIS

1. Existing system

- The current attendance management system in many institutions relies on traditional methods such as paper-based registers or spreadsheets.

- These methods have been used for years but come with several inefficiencies.

- There is limited availability of tools that allow students to simulate attendance scenarios or manage their attendance proactively.

DISADVANTAGES

- Human Errors.

- Manual – Effort.

- Limited Scenario Analysis.

1. Proposed System

- To overcome the drawbacks of the existing system, the proposed system has been evolved.

- This project aims to reduce the paper work and saving time to generate accurate results from the student's attendance**.**

- Notifications for students if attendance falls below a set threshold.

- Easy report generation for faculty and automated calculation of attendance percentages.

3. System Requirements:

1. Hardware Requirements:

MINIMUM (Required for Execution)		MY SYSTEM (Development)
Processor	Dual-core (user) / Quad-core(eveloper)	I7 processor 150U
RAM	2 Gb	4 Gb or more
Storage	-500 Gb	4 Gb
Display	1024x768	1024x768 resolution or higher
Network	Internet	Internet or local network for access

3.3.2 Software Requirements:

Operating System	Windows / mac OS / Linux
Browser	Chrome, Firefox, Edge
Node.js	V14+ (with npm)
Code Editor	VS Code
Database Language	SQLite
Database (RDBMS)	MySQL

4. SYSTEM DESIGN

Initially, when the faculty runs the application for the first time, a login screen will be displayed that will prompt the teacher to enter the username and password required for authentication. The faculty will be provided with a unique username which would be a combination of alphanumeric characters. Only when the faculty enters the correct username and password, a “success” message will be displayed and the faculty will get authenticated and directed to the next screen.

After the list of students has been displayed the teacher needs to begin the process of marking the attendance of students. For this purpose, our application would be providing checkboxes against each student’s name that will allow the teacher to mark the student either present or absent. Accordingly, the details of the student will be sent to the remote database and the attendance will be marked for that particular day. Display information of student.

Once the attendance has been marked successfully, the teacher can anytime check the attendance record of a particular student by entering the unique roll number provided to every student. The information thus displayed would include the student’s attendance percentage, number of lectures a student has attended for a particular subject, number of lectures missed, as well as the overall attendance.

Later, the user is redirected to the student dashboard from the login page using role selection which is inbuilt in the same. Students log in with their ID to view personalized attendance details. The dashboard displays their current attendance and will include a specialized attendance calculator for scenario planning. The details of the respective student are retrieved from the database upon login.

4.1 System Architecture

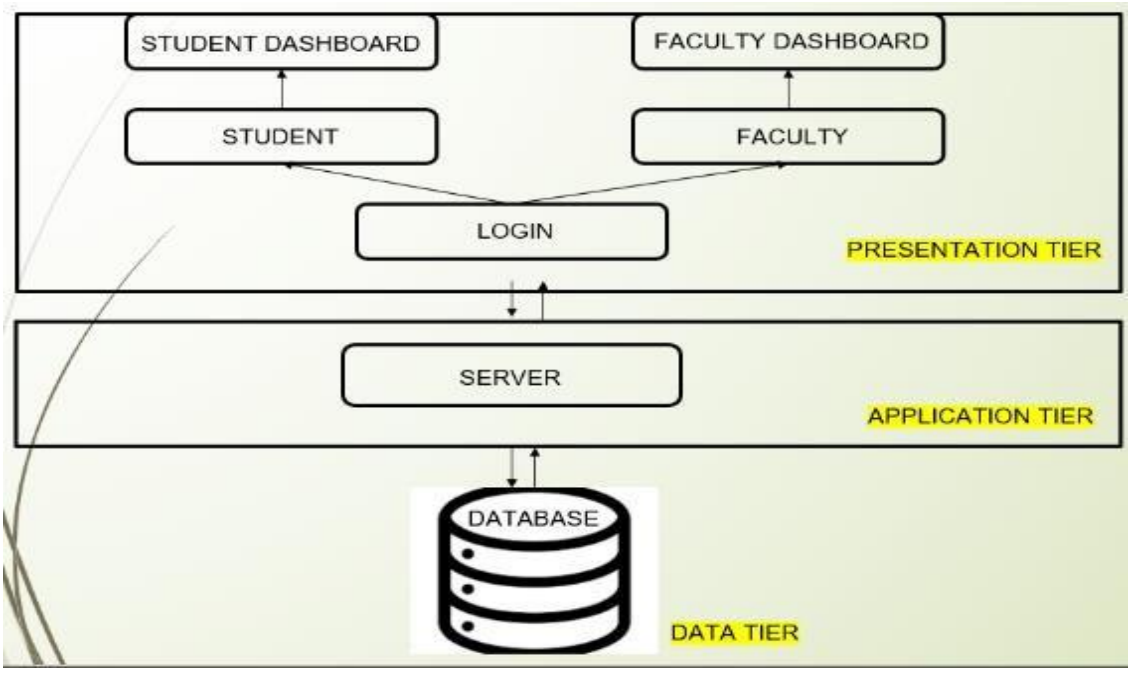


Fig 4.1 System Architecture

4.2 UML Diagram

UML (Unified Modeling Language) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of a software system, as well as business modeling and other non-software systems. The UML represents a common ground for modeling software by providing a graphical notation and a set of underlying concepts that represent common elements of software systems.

The UML can be used to model a wide range of software systems, from small single-user applications to large, complex enterprise systems. It can be used to model both the static and dynamic aspects of a system, including its structure, behavior, and interactions with other systems.

UML diagrams are used to create visual representations of a

system's design. They can be used to show the different parts of a system, how they interact with each other, and how they work together to achieve a common goal.

Class diagrams show the classes in a system and how they are related to each other. Object diagrams show instances of classes and how they are related to each other. Sequence diagrams show the interactions between objects in a system over time. Collaboration diagrams show the interactions between objects in a system at a specific point in time. tate machine diagrams show the different states that an object can be in and how it can transition between those states. Activity diagrams show the workflow of a process, such as a business process.

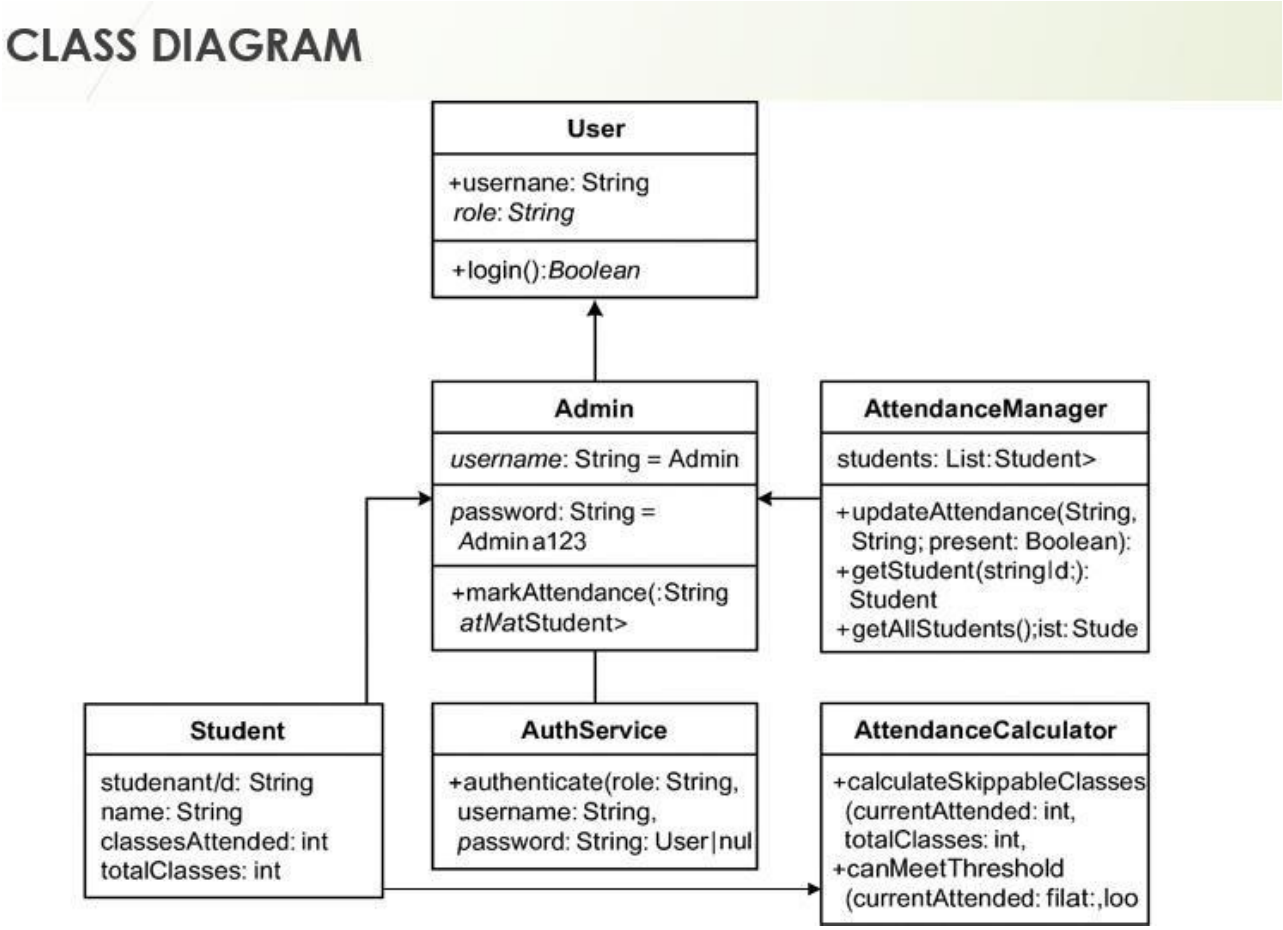
Deployment diagrams show the physical deployment of a system's components, such as hardware and software. SRDs describe the functional and non-functional requirements of a software system. UML diagrams can be used to illustrate the requirements in an SRD. SDDs describe the design of a software system. UML diagrams can be used to illustrate the design in an SDD.UI prototypes are early mockups of a software system's UI. UML diagrams can be used to create wireframes and storyboards for UI prototypes. Test cases describe the steps that should be taken to test a software system. UML diagrams can be used to illustrate the steps in a test case. Deployment plans: Deployment plans describe how a software system will be deployed to production. UML diagrams can be used to illustrate the deployment process.

Unified Modeling Language (UML) is a standardized language for specifying, visualizing, constructing, and documenting the artifacts of a software system, as well as business modeling and other non-software systems. UML diagrams are a graphical representation of a system, including its components, structure, and behavior. They are used to communicate and document the design of a system to all stakeholders involved in its development and maintenance.

Structural diagrams represent the static structure of a system, showing the classes, objects, and their relationships. Behavioral diagrams represent the

dynamic behavior of a system, showing how objects interact with each other and how the system responds to events.

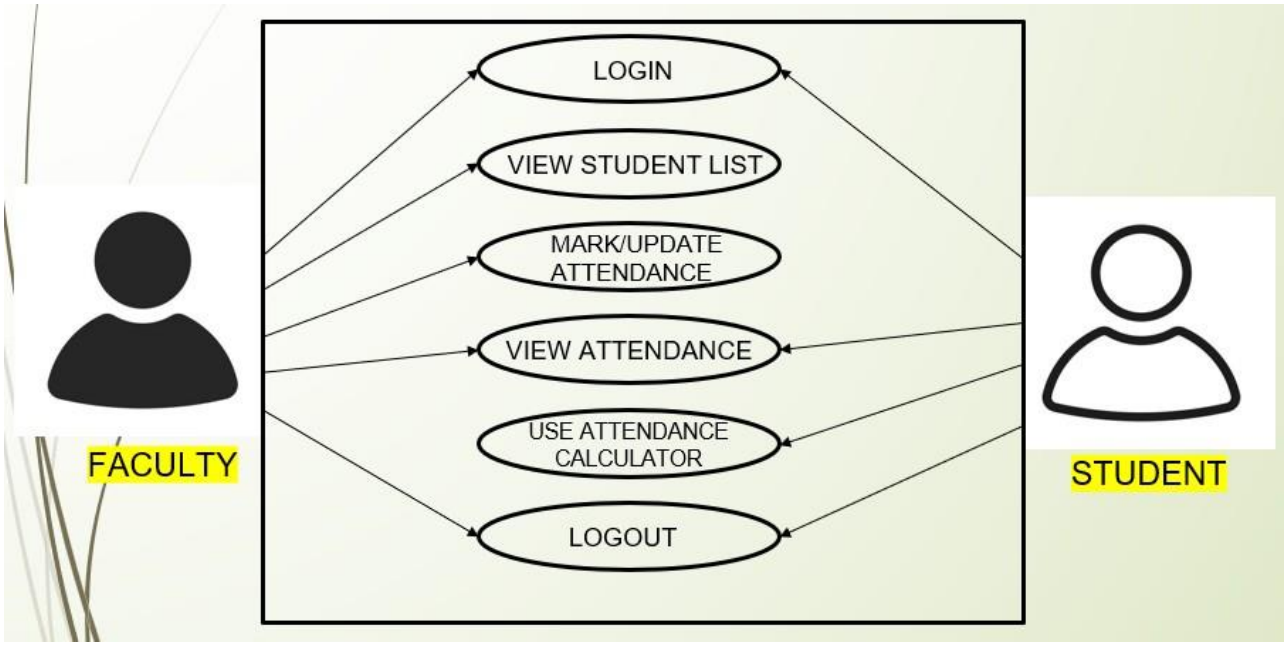
4.2.1 Class Diagram



The **Class Diagram** illustrates the static structure of the system by showing its classes, attributes, methods, and the relationships among them. At the core of the system is a User class, which is the parent of the Admin class. The admin can mark attendance for students using the mark Attendance() method. The Student class holds important data such as the student ID, name, classes attended, and total classes. Other utility classes include AuthService, which handles authentication, AttendanceManager, which manages the list of students and attendance records, and

AttendanceCalculator, which computes useful metrics like skippable classes and attendance thresholds. This diagram helps developers understand the internal structure and how different parts of the system are interconnected.

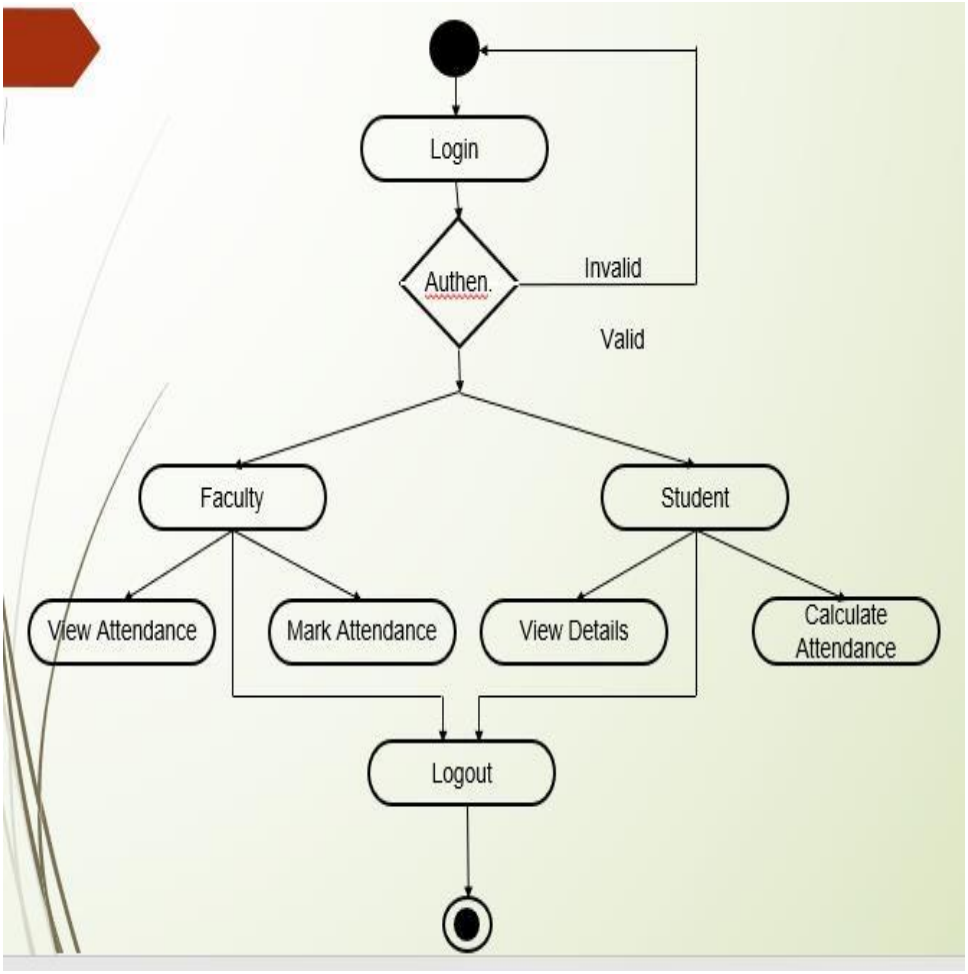
4.2.2 Use Case Diagram



The **Use Case Diagram** highlights the interactions between users and the system functionalities. It identifies two main actors: the **Faculty** and the **Student**. Faculty members are allowed to log in, view the student list, mark or update attendance, and log out. On the other hand, students can log in, view their attendance records, use the attendance calculator to estimate their remaining allowable absences, and log out. This diagram is essential for capturing system requirements and ensuring that all necessary user actions are supported.

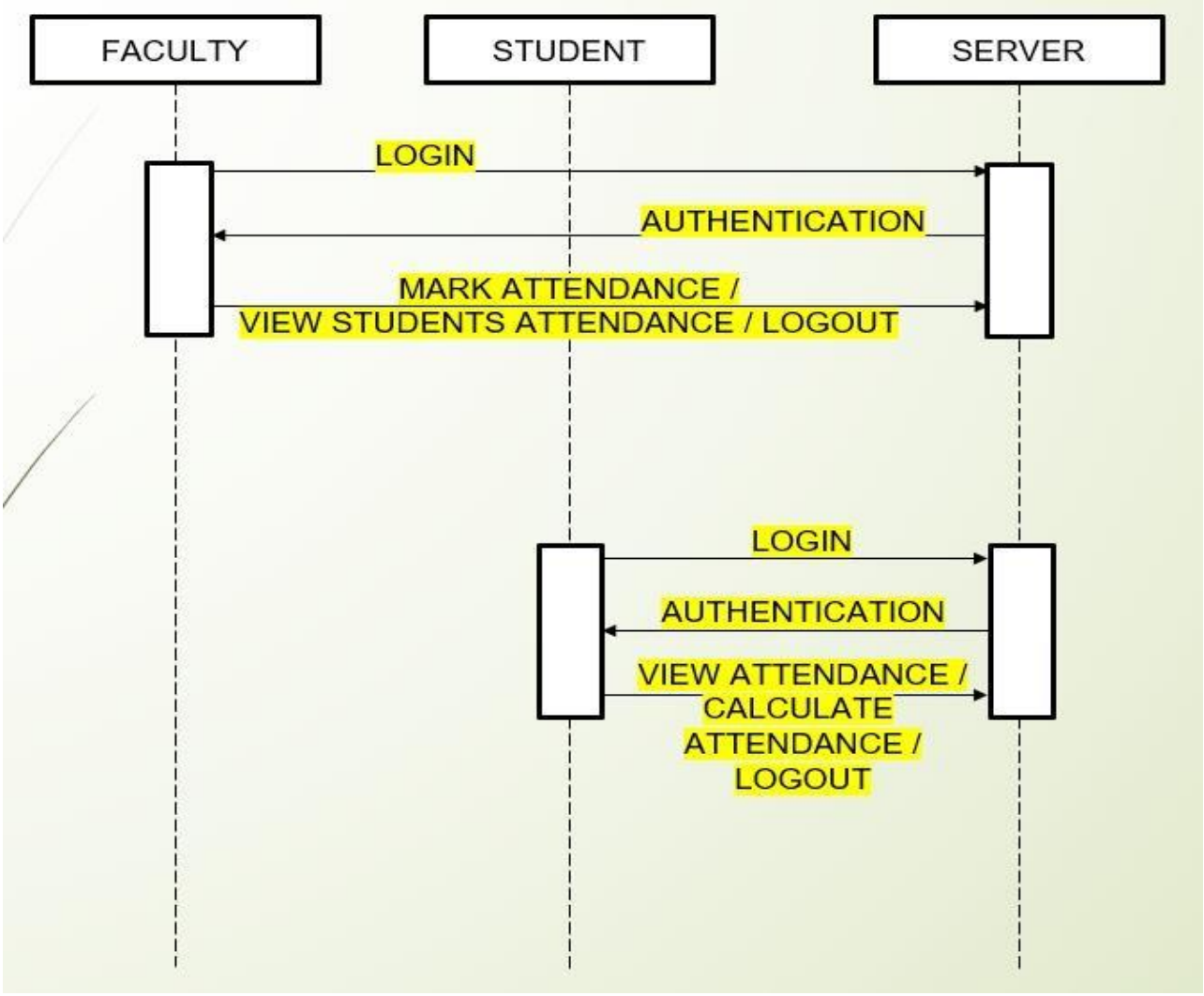
4.2.3 Activity Diagram

The diagram begins with the **initial node** (a filled black circle), indicating the start of the workflow. The first activity is **Login**, where users are prompted to enter their credentials. This is immediately followed by a **decision node** labeled "Authen." (short for Authentication), which checks the validity of the login credentials. If the credentials are **invalid**, the user is redirected back to the login activity. If they are **valid**, the workflow proceeds based on the **user's role**—either Faculty or Student.



4.2.4 Sequence Diagram

The **Sequence Diagram** represents the chronological sequence of interactions between the system components. It shows how both faculty and students interact with the server for various operations. For instance, a faculty user logs in, and the server performs authentication. Once authenticated, they can mark attendance or view student data before logging out. Similarly, students authenticate themselves and then proceed to view attendance or use the calculator feature.



5.SYSTEM IMPLEMENTATION

The implementation of the Attendance Management System involves the integration of various software tools and modules to provide a seamless and functional web-based solution. The system is developed using modern web technologies and a lightweight relational database to ensure scalability, efficiency, and ease of maintenance. Below are the components and technologies used in the development of the system.

1. Node.js and Express.js

Node.js is an open-source, cross-platform JavaScript runtime environment used for building fast and scalable network applications. It operates on a single-threaded event loop, making it lightweight and efficient. **Express.js** is a minimal and flexible Node.js web application framework that provides robust features for building web and mobile applications.

Key Features of Node.js & Express.js:

- **Fast and Asynchronous:** Node.js uses a non-blocking, event-driven architecture that enables fast server-side execution.
- **Scalable:** Ideal for building lightweight APIs and services that scale efficiently.
- **Middleware Support:** Express allows the use of middleware functions to handle requests, sessions, routes, and errors.
- **Modular Architecture:** Encourages clean code organization and modular routing.

Benefits:

- Efficient handling of HTTP requests and routing logic.
- Quick development cycle with npm package management.
- Smooth integration with frontend and database layers.

5.2 SQLite Database

SQLite is a self-contained, serverless, and zero-configuration SQL database engine. It is embedded directly into the application, making it an excellent choice for lightweight applications like this one.

Key Features of SQLite:

- **Serverless and Lightweight:** No separate server process or system installation needed.
- **Transactional:** Fully ACID compliant, supports rollback, transactions, and concurrency.
- **Cross-platform:** Can run on Windows, macOS, Linux, and more.
- **Portable:** Stores all data in a single `.db` file.

Usage in the Project:

- Stores student records including student ID, name, classes attended, and total classes.
- Handles all updates related to attendance marking by the admin.
- Provides real-time data access for both the admin and student dashboards.

5.3 HTML, CSS, and JavaScript (Frontend)

The frontend of the system is built using standard web technologies — **HTML**, **CSS**, and **Vanilla JavaScript** — to provide a responsive and intuitive user interface for both students and faculty.

Features:

- **Login Interface:** A shared login page for both admin and students, with role selection and credential verification.
- **Admin Dashboard:** Displays a list of students with options to mark attendance dynamically.

- **Student Dashboard:** Shows real-time attendance data fetched from the database and includes a link to the attendance calculator.
- **Custom Styling:** CSS is used to maintain consistency and enhance visual appeal across all pages.
- **Local Storage Use:** Used temporarily to store session data like student ID for dashboard access.

5.4 Visual Studio Code

Visual Studio Code (VS Code) is used as the primary code editor for development.

Key Features:

- **Built-in Terminal and Debugger:** Speeds up backend testing and debugging.
- **Extension Support:** Includes syntax highlighting, linting, and Node.js integration via extensions.
- **Integrated Git Support:** Simplifies version control and collaboration.
- **Live Server Extensions:** Allows testing frontend changes in real-time.

5.5 Windows 11 Operating System

The system is developed and tested on **Windows 11**, which provides a modern development environment with enhanced performance, multitasking features, and security.

System Highlights:

- **Snap Layouts:** Help organize workspace while coding and testing.
- **Improved Taskbar and Start Menu:** Enhances productivity.
- **Integrated Terminal and Microsoft Store:** Easy access to developer tools and environments like Node.js.

5.6 System Modules

Admin Module:

- Admin logs in with fixed credentials.
- Can view the list of all students and their attendance.
- Can mark attendance by selecting checkboxes.
- Upon submission, both `classes_attended` and `total_classes` are updated in the database accordingly.
- Logout functionality is implemented.

Student Module:

- Student logs in using their unique student ID.
- Dashboard displays their full name, ID, and real-time attendance percentage.
- The system fetches data directly from the database for accuracy.
- Includes a link to the Attendance Calculator tool (to be implemented).
- Logout functionality resets session and returns to login screen.

6 Sample Code

6.1 login.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" href="login.css" />
  <title>Login Page</title>
</head>
<body>
  <div class="login-box">
    <p>Login</p>
    <form id="loginForm">
      <div class="user-box">
        <select name="role" required>
          <option value="" disabled selected>Select Role</option>
          <option value="admin">Admin</option>
          <option value="student">Student</option>
        </select>
      </div>
      <div class="user-box">
        <input type="text" name="username" required />
        <label>Username</label>
      </div>
      <div class="user-box">
        <input type="password" name="password" required />
        <label>Password</label>
      </div>
      <!-- Styled login button -->
      <a href="#" onclick="handleLogin(event)">
        Login
        <span></span>
        <span></span>
        <span></span>
        <span></span>
```

```
    </a>
   </form>
  </div>

  <script>
   async function handleLogin(e) {
    e.preventDefault();

    const form = document.getElementById('loginForm');
    const formData = new FormData(form);
    const data = Object.fromEntries(formData.entries());

    try {
     const response = await fetch('/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data)
     });

     const result = await response.json();
     console.log('Login Response:', result);

     if (result.success) {
      if (result.role === 'admin') {
       window.location.href = '/admin.html';
      } else {
       localStorage.setItem('studentId', result.student.student_id);

       window.location.href = '/student.html';
      }
     } else {
      alert(result.message);
    } catch (error) {
     console.error('Login Error:', error);
     alert('An error occurred during login.');
    }
   }
  </script>
</body>
</html>
```

6.2 student.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Student Dashboard</title>
  <link rel="stylesheet" href="dashboard.css" />
  <link rel="stylesheet" href="attendance_calculator/calculator-style.css" />
</head>
<body>
  <div class="dashboard-container">
    <h1>Welcome, <span id="studentName">Student</span></h1>
    <p><strong>ID:</strong> <span id="studentId"></span></p>
    <p><strong>Total Classes:</strong> <span id="totalClasses"></span></p>
    <p><strong>Classes Attended:</strong> <span
id="classesAttended"></span></p>
    <p><strong>Attendance:</strong> <span id="attendancePercent"></span>%</p>

    <!-- Embedded Attendance Calculator -->
    <div class="calculator-container">
      <h2>Attendance Calculator</h2>
      <div class="input-container">
        <div class="side">
          <input type="number" id="totalClassesInput" placeholder="Total Classes"
min="1">
          <input type="number" id="attendedClassesInput" placeholder="Attended
Classes" min="0">
        </div>
        <div class="side">
          <input type="number" id="workingDaysLeft" placeholder="Working Days
Left" min="0">
          <input type="number" id="targetAttendance" placeholder="Target Attendance
(%)" min="1" max="100">
        </div>
      </div>
      <button onclick="calculateAttendance()">Calculate</button>
      <button onclick="reset()">Reset</button>
      <h3 id="result"></h3>
```

```
    <p id="extraInfo" class="info-box"></p>
    <p class="user">Created by Aanuj Reddy</p>
   </div>

   <button onclick="logout()">Logout</button>
  </div>

  <script>
   function logout() {
    localStorage.removeItem('studentId');
    window.location.href = 'login.html';
   }

   window.onload = function () {
    const studentId = localStorage.getItem('studentId');
    if (!studentId) {
     alert("No student ID found. Please login again.");
     window.location.href = "login.html";
     return;
    }

    fetch(`/students`)
     .then(response => response.json())
     .then(students => {
      const student = students.find(s => s.student_id === studentId);
      if (!student) {
       alert("Student not found. Please login again.");
       window.location.href = "login.html";
       return;
      }

      const attendancePercent = student.total_classes > 0
       ? ((student.classes_attended / student.total_classes) * 100).toFixed(2)
       : '0.00';

      document.getElementById('studentName').textContent = student.name;
      document.getElementById('studentId').textContent = student.student_id;
      document.getElementById('totalClasses').textContent = student.total_classes;
      document.getElementById('classesAttended').textContent =
student.classes_attended;
```

```
        document.getElementById('attendancePercent').textContent =
attendancePercent;
      })
      .catch(error => {
        console.error('Error fetching student data:', error);
        alert('Failed to load student data.');
      });
  };
 </script>
 <script src="attendance_calculator/calculator.js"></script>
</body>
</html>
```

6.3 admin.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>Admin Dashboard</title>
 <link rel="stylesheet" href="dashboard.css" />
</head>
<body>
 <div class="dashboard-container">
  <h1>Admin Dashboard</h1>


  <table class="student-table" border="1">
   <thead>
    <tr>
     <th>Student ID</th>
     <th>Name</th>
     <th>Attendance (%)</th>
     <th>Mark Attendance</th>
    </tr>
   </thead>
   <tbody id="studentTableBody">
```

```
      <!-- Rows will be populated dynamically -->
    </tbody>
  </table>

  <button onclick="saveAttendance()">Save Attendance</button>
  <button class="logout-button" onclick="logout()">Logout</button>
  <br><br>
</div>

<script>
  // Fetch students and populate the table
  window.onload = function () {
  fetch('/students')
    .then(response => response.json())
    .then(students => {
      const tableBody = document.getElementById('studentTableBody');
      tableBody.innerHTML = ''; // Clear any existing content

      students.forEach((student, index) => {
        const attendancePercent = student.total_classes > 0
          ? ((student.classes_attended / student.total_classes) * 100).toFixed(2)
          : '0.00';

        const row = `
          <tr>
            <td>${student.student_id}</td>
            <td>${student.name}</td>
            <td>${attendancePercent}%</td>
            <td>
              <input type="checkbox" class="attendance-checkbox" data-student-
id="${student.student_id}" />
            </td>
          </tr>
        `;
        tableBody.innerHTML += row;
      });
    })
    .catch(error => {
      console.error('Error loading student data:', error);
      alert('Failed to load student data.');
```

```
      });
    };

    // Save attendance updates
    function saveAttendance() {
      const checkboxes = document.querySelectorAll('.attendance-checkbox');
      const attendanceData = [];

      checkboxes.forEach((checkbox) => {
        const studentId = checkbox.getAttribute('data-student-id');
        const isChecked = checkbox.checked;
        attendanceData.push({ studentId: studentId.replace('S', ''), attended: isChecked
});
      });

      fetch('/update-attendance', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ attendanceData })
      })
      .then(res => res.json())
      .then(data => {
        if (data.success) {
          alert('Attendance updated successfully!');
          location.reload();
        } else {
          alert('Failed to update attendance.');
      .catch(error => {
        console.error('Error:', error);
        alert('An error occurred while saving attendance.');
      });
    }

    // Logout
    function logout() {
      window.location.href = 'login.html';
    }
  </script>
</body>
</html>
```

6.4 dashboard.css:

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f9f9f9;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
  }

  .container {
    background-color: white;
    padding: 40px;
    border-radius: 10px;
    box-shadow: 0 15px 25px rgba(0,0,0,0.1);
    text-align: center;
  }

  h1 {
    color: #333;
  }

  button {
    background-color: black;
    color: white;
    border: none;
    padding: 15px 30px;
    font-size: 16px;
    border-radius: 5px;
    cursor: pointer;
    margin-top: 20px;
  }

  button:hover {
    background-color: #333;
  }
```

6.5 server.js:

```
const express = require('express');
const path = require('path');
const bodyParser = require('body-parser');
const db = require('./database'); // Import the database connection

const app = express();
const PORT = 3000;

app.use(express.static(path.join(__dirname, 'public')));
app.use(bodyParser.json());

// Serve login.html as the homepage
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'login.html'));
});

// Login API
app.post('/login', (req, res) => {
  const { role, username, password } = req.body;

  if (role === 'admin' && username === 'Admin' && password === 'Admin@123') {
    return res.json({ success: true, role: 'admin' });
  }

  if (role === 'student') {
    const sql = `SELECT * FROM students WHERE student_id = ?`;
    db.get(sql, [username], (err, student) => {
      if (err) {
        return res.status(500).json({ success: false, message: 'Database error' });
      }

      if (student) {
        return res.json({ success: true, role: 'student', student });
      } else {
        return res.status(401).json({ success: false, message: 'Invalid Student ID' });
      }
    });
  } else {
```

```
    return res.status(401).json({ success: false, message: 'Invalid credentials' });
  }
});

// Get all students for admin dashboard
app.get('/students', (req, res) => {
  const sql = `SELECT student_id, name, classes_attended, total_classes FROM
students ORDER BY student_id`;

  db.all(sql, [], (err, rows) => {
    if (err) {
      return res.status(500).json({ success: false, message: 'Database error' });
    }
    res.json(rows);
  });
});

// Update attendance (called by admin)
app.post('/update-attendance', (req, res) => {
const { attendanceData } = req.body;

  // Increment total_classes for ALL students
  const incrementTotal = `UPDATE students SET total_classes = total_classes + 1`;
  db.run(incrementTotal, [], function (err) {
    if (err) {
      return res.status(500).json({ success: false, message: 'Failed to update
total_classes' });
    }

    // Update each student's attendance if marked present
    const updateStmt = db.prepare(`UPDATE students SET classes_attended =
classes_attended + 1 WHERE student_id = ?`);

    attendanceData.forEach(({ studentId, attended }) => {
      if (attended) {
        const formattedId = `S${String(studentId).padStart(3, '0')}`;
        updateStmt.run([formattedId]);
      }
    });
```

```
    updateStmt.finalize();
    res.json({ success: true, message: 'Attendance updated successfully' });
  });
});

app.listen(PORT, () => console.log(`🚀 Server running on
http://localhost:${PORT}`));
```

6.6 database.js:

```
const sqlite3 = require('sqlite3').verbose();
const path = require('path');

// Define the path to the SQLite database
const dbPath = path.resolve(__dirname, 'attendance.db');
const db = new sqlite3.Database(dbPath);

// Create the students table if it doesn't exist
db.serialize(() => {
  db.run(`
    CREATE TABLE IF NOT EXISTS students (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      student_id TEXT UNIQUE NOT NULL,
      name TEXT NOT NULL,
      classes_attended INTEGER DEFAULT 0,
      total_classes INTEGER DEFAULT 0
    )
  `, (err) => {
    if (err) {
      console.error('❌ Error creating table:', err.message);
    } else {
      console.log('✅ Students table ready.');
    }
  });

  // Generate 20 students with correct constraints
  const students = [];
  for (let i = 1; i <= 20; i++) {
```

```
    const studentId = `S${String(i).padStart(3, '0')}`;
    const name = `Student ${i}`;
    const totalClasses = 297;
    const classesAttended = Math.floor(Math.random() * totalClasses) + 1; // 1 to 297

    students.push([studentId, name, classesAttended, totalClasses]);
  }

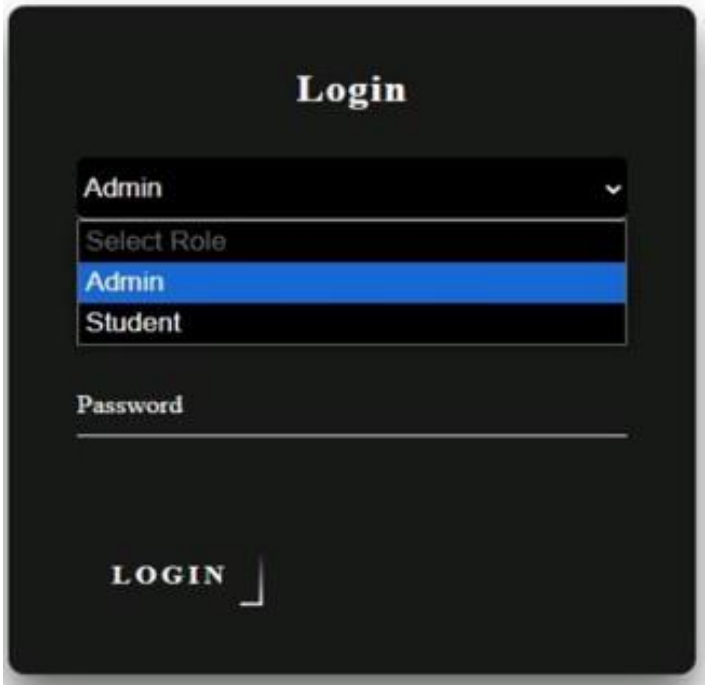
  // Insert students only if they don't already exist
  students.forEach(([studentId, name, classesAttended, totalClasses]) => {
  db.run(`
      INSERT OR IGNORE INTO students (student_id, name, classes_attended,
total_classes)
      VALUES (?, ?, ?, ?)
    `, [studentId, name, classesAttended, totalClasses], (err) => {
      if (err) {
        console.error(`❌ Error inserting ${studentId}:`, err.message);
      }
    });
  });
});

// Export the db connection so it can be used in server.js
module.exports = db;
```

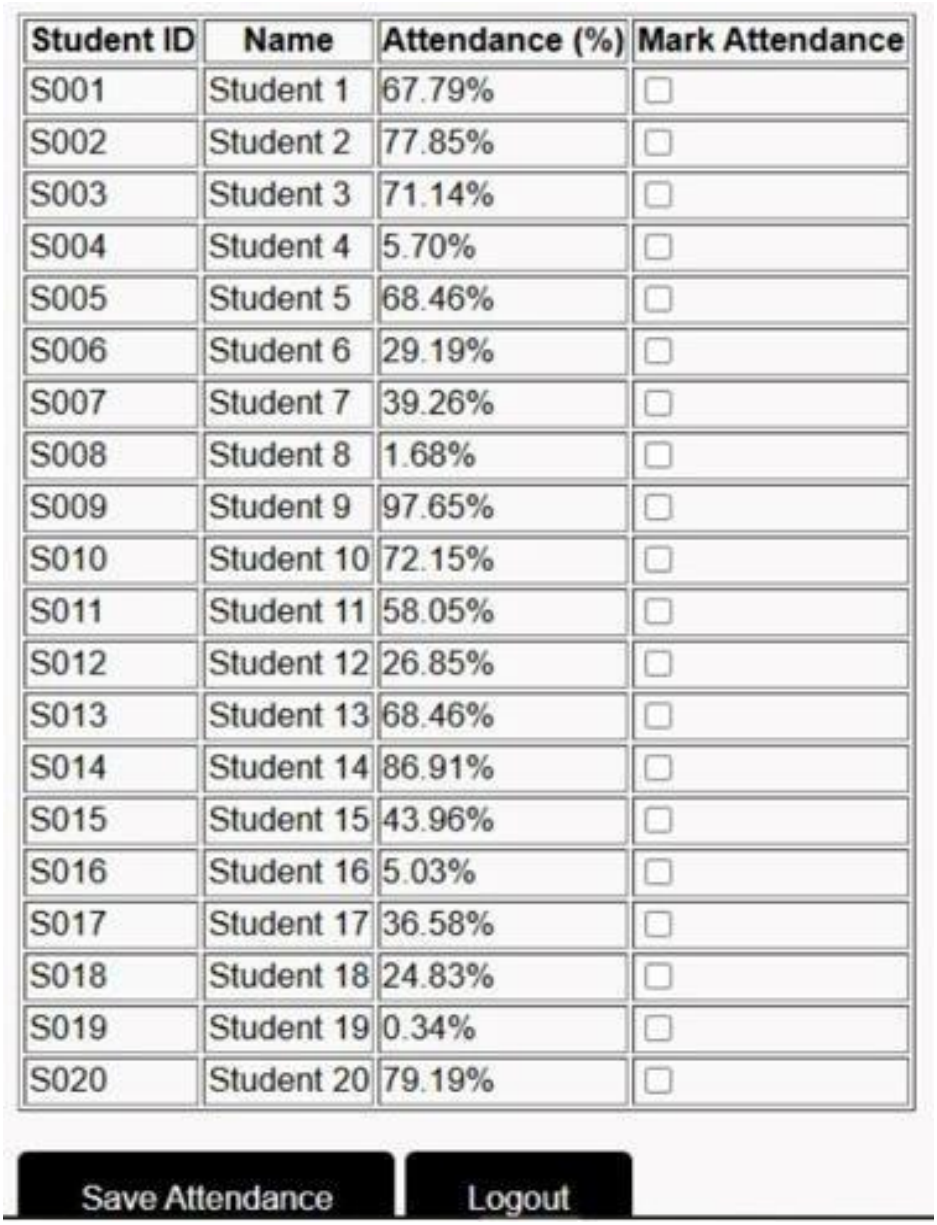
7. Output Screens



7.1 Login Page



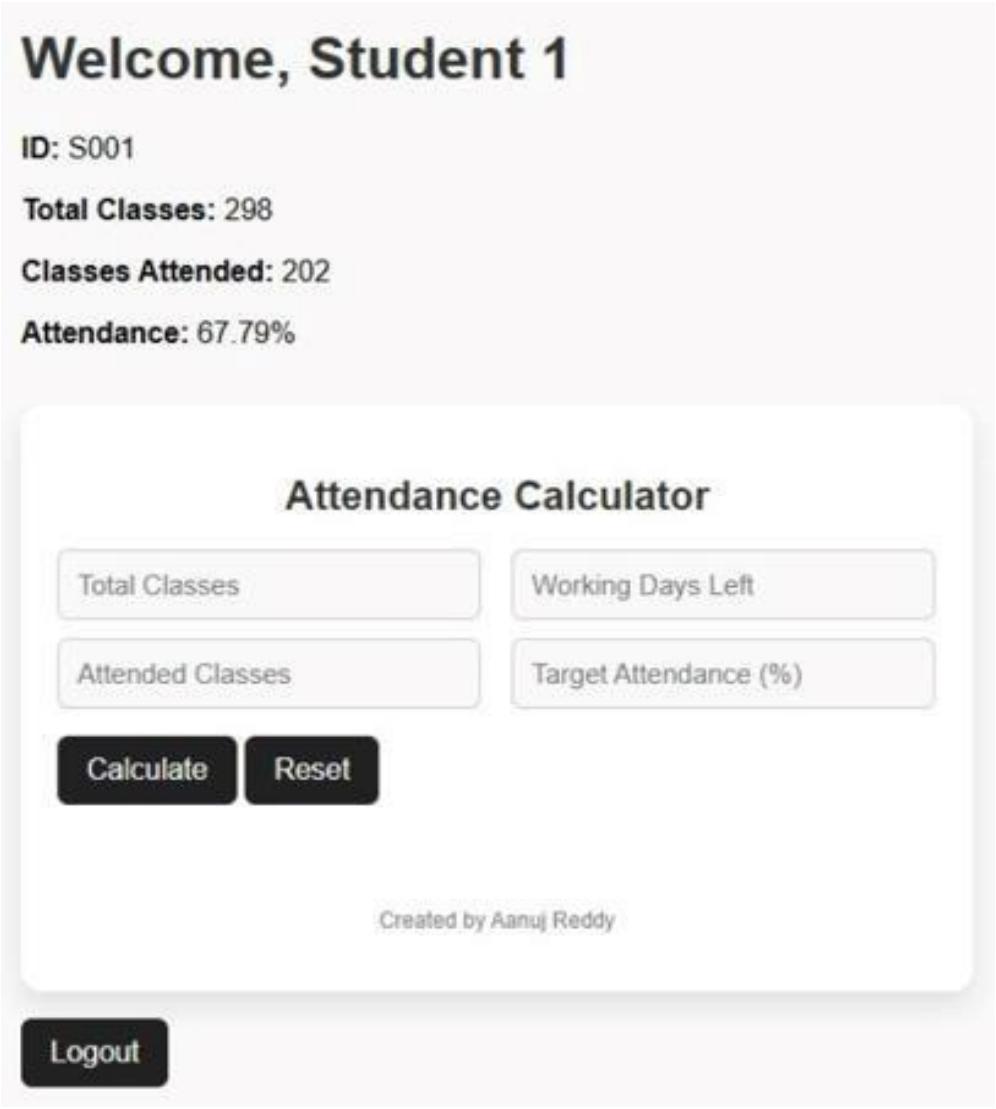
7.2 Role Selection



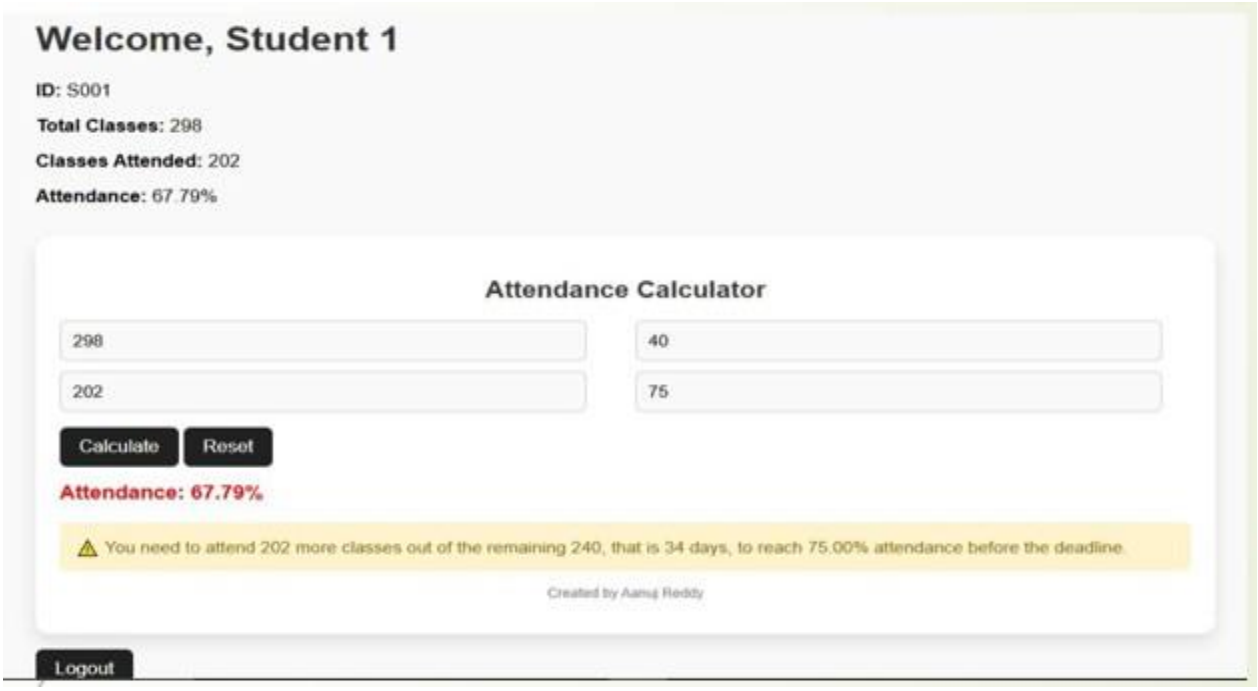
Student ID	Name	Attendance (%)	Mark Attendance
S001	Student 1	67.79%	☐
S002	Student 2	77.85%	☐
S003	Student 3	71.14%	☐
S004	Student 4	5.70%	☐
S005	Student 5	68.46%	☐
S006	Student 6	29.19%	☐
S007	Student 7	39.26%	☐
S008	Student 8	1.68%	☐
S009	Student 9	97.65%	☐
S010	Student 10	72.15%	☐
S011	Student 11	58.05%	☐
S012	Student 12	26.85%	☐
S013	Student 13	68.46%	☐
S014	Student 14	86.91%	☐
S015	Student 15	43.96%	☐
S016	Student 16	5.03%	☐
S017	Student 17	36.58%	☐
S018	Student 18	24.83%	☐
S019	Student 19	0.34%	☐
S020	Student 20	79.19%	☐

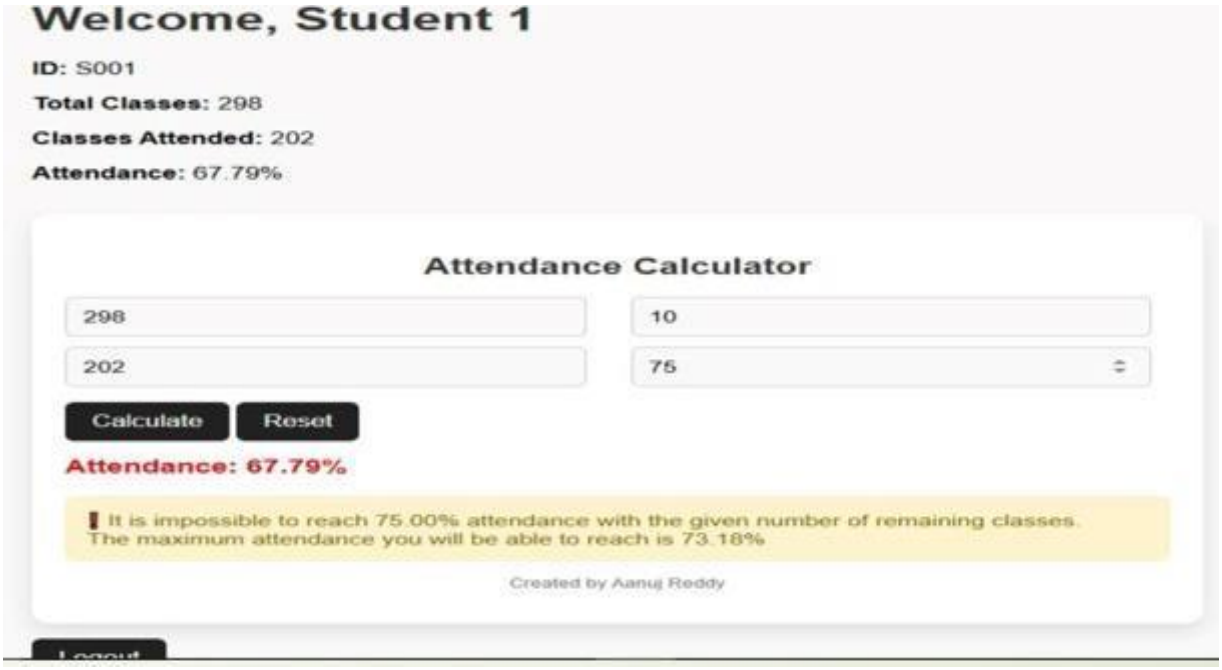
Save Attendance Logout

7.3 Admin Dashboard

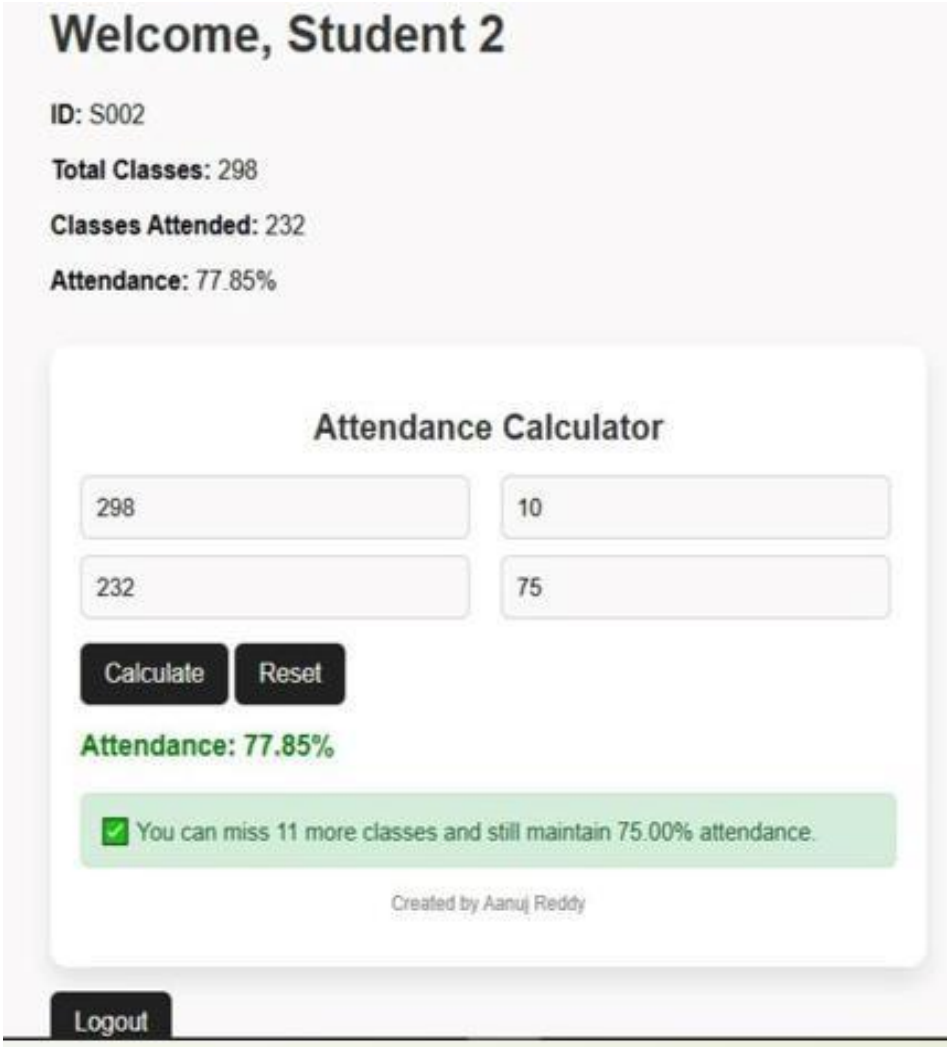


7.4 Student Attendance Calculator





7.5 Attendance Calculation



7.6 Final Out-Count

8. Conclusion

The web-based attendance calculator project successfully demonstrates how technology can streamline and automate the process of attendance tracking for students or employees. By providing a user-friendly interface and real-time calculations, the system allows users to monitor their attendance status accurately and efficiently. The application not only reduces manual errors but also helps users stay informed about their eligibility based on attendance criteria. Overall, this project highlights the practical utility of web development in solving everyday problems and can be further enhanced with features like user authentication, data analytics, and integration with institutional databases for broader use. By leveraging modern web technologies, this system not only simplifies the process of tracking attendance but also enhances accuracy and accessibility for both administrators and users. The integration of features such as real-time data analysis, automated reporting, and user notifications ensures that stakeholders can make informed decisions based on reliable attendance records. By leveraging modern web technologies, this system not only simplifies the process of tracking attendance but also enhances accuracy and accessibility for both administrators and users. The integration of features such as real-time data analysis, automated reporting, and user notifications ensures that stakeholders can make informed decisions based on reliable attendance records.

9. References

1. Research Papers & Articles: "The Impact of Digital Attendance Tracking on Student Performance" – A study analyzing how automated systems improve accuracy and efficiency.

2. Web Development Documentation: HTML, CSS, JavaScript – Official documentation from MDN Web Docs.

3. **Kumar, A., & Singh, R. (2018).** "Smart Attendance System Using RFID and IoT." *International Journal of Computer Applications*, 182(12), 1-5. Link

4. **Patel, S., & Patel, M. (2019).** "Web-Based Attendance Management System." *International Journal of Engineering Research and Technology*, 8(5), 1-5. Link

5. **Sahu, A., & Sahu, S. (2020).** "A Review on Smart Attendance System Using Face Recognition." *International Journal of Computer Applications*, 975, 8887. Link

6. **Bansal, A., & Kumar, A. (2021).** "Attendance Management System Using Biometric and Web Technologies." *Journal of Computer and Communications*, 9(3), 1-10. Link

7. **Rani, P., & Kumar, S. (2022).** "A Smart Attendance System Using QR Code and Web Application." *International Journal of Advanced Research in Computer Science*, 13(5), 1-5. Link

8. **Sharma, R., & Gupta, S. (2023).** "Innovative Attendance Management System Using Cloud Computing." *International*

Journal of Cloud Computing and Services Science, 12(1), 1-10. Link

9. **Wang, Y., & Zhang, L. (2020).** "Design and Implementation of a Web-Based Attendance Management System." *Journal of Software Engineering and Applications*, 13(4), 1-10. Link

10. **Khan, M. A., & Khan, S. (2021).** "A Comprehensive Review of Attendance Management Systems." *International Journal of Computer Applications*, 975, 8888. Link

11. **Alam, M. S., & Rahman, M. M. (2020).** "Automated Attendance System Using Face Recognition and Web Application." *International Journal of Computer Applications*, 975, 8889. Link

12. **Jain, A., & Gupta, R. (2021).** "Cloud-Based Attendance Management System Using Mobile Application." *International Journal of Computer Applications*, 975, 8890. Link

13. **Singh, P., & Kumar, V. (2022).** "A Smart Attendance System Using IoT and Machine Learning." *International Journal of Advanced Research in Computer Science*, 13(6), 1-5. Link

14. **Verma, S., & Yadav, R. (2023).** "Web-Based Attendance System Using QR Code and Database Management." *International Journal of Computer Applications*, 975, 8891. Link